AgentCart

> **Make any merchant transactable by an AI buyer.**

AgentCart is an agentic commerce platform that enables an AI buyer to discover a merchant's products, understand the catalog, make recommendations, create a purchase intent, and complete a bounded transaction through Razorpay.

The system is divided into two primary services:

- **AI Service** — Python-based intelligence and agent layer
- **Backend Service** — Spring Boot-based commerce, business logic, and Razorpay layer

---

1. Core Idea

Traditional commerce:

```
Human
  ↓
Website
  ↓
Browse
  ↓
Cart
  ↓
Checkout
  ↓
Payment
```

Agentic commerce:

```
Human
  ↓
AI Buyer
  ↓
Merchant AI Gateway
  ↓
Product Discovery
  ↓
Recommendation
  ↓
```

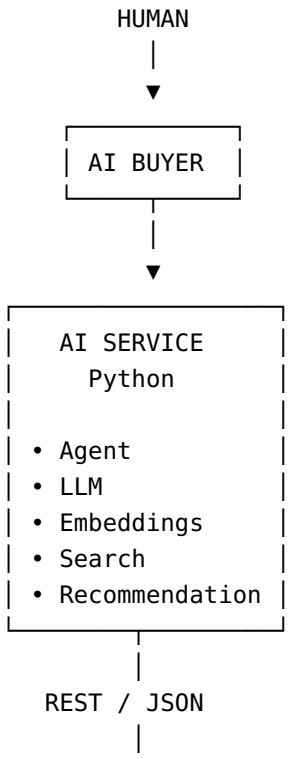
```
Purchase Intent
  ↓
Policy / Authorization
  ↓
Razorpay
  ↓
Payment
```

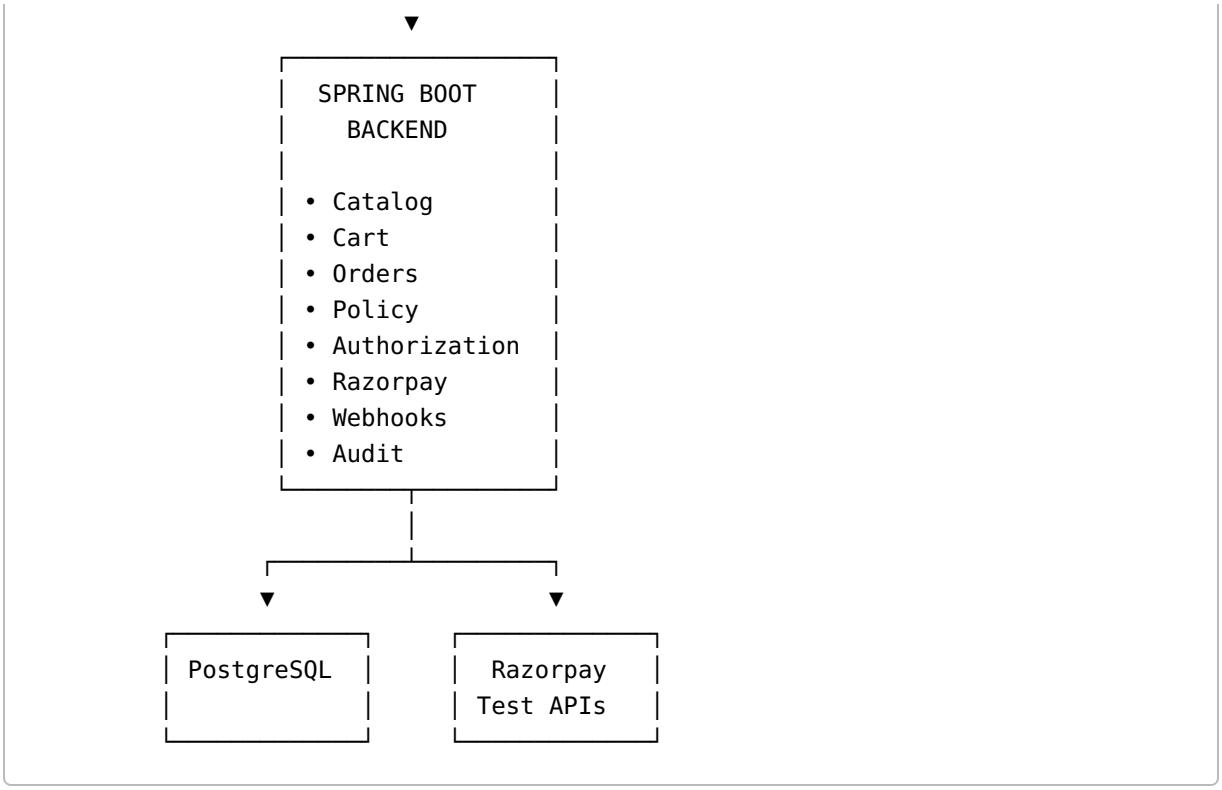
The AI should **never directly execute a financial action**.

```
AI
 ↓
Request / Decision
 ↓
Spring Boot
 ↓
Validation + Policy
 ↓
Razorpay
```

---

2. High-Level Architecture





---

3. Repository Structure

Recommended monorepo:

```
agentcart/
│
├── README.md
├── .gitignore
├── docker-compose.yml
│
├── ai-service/
│   ├── README.md
│   ├── requirements.txt
│   ├── .env.example
│   │
│   ├── app/
│   │   ├── main.py
│   │   │
│   │   ├── agents/
│   │   │   ├── buyer_agent.py
│   │   │   └── prompts.py
│   │   │
```

```
│   │   ├── llm/
│   │   │   └── client.py
│   │   │
│   │   ├── embeddings/
│   │   │   └── embedder.py
│   │   │
│   │   ├── search/
│   │   │   └── product_search.py
│   │   │
│   │   ├── recommendations/
│   │   │   └── recommender.py
│   │   │
│   │   ├── tools/
│   │   │   ├── catalog_tools.py
│   │   │   ├── cart_tools.py
│   │   │   └── purchase_tools.py
│   │   │
│   │   └── schemas/
│   │       └── agent_schemas.py
│   │
│   └── tests/
│
├── backend/
│   ├── README.md
│   ├── pom.xml
│   ├── .env.example
│   │
│   └── src/
│       ├── main/
│       │   └── java/
│       │       └── com/
│       │           └── agentcart/
│       │
│       │               ├── AgentCartApplication.java
│       │               │
│       │               ├── controller/
│       │               ├── service/
│       │               ├── repository/
│       │               ├── entity/
│       │               ├── dto/
│       │               ├── razorpay/
│       │               ├── policy/
│       │               ├── audit/
│       │               └── config/
│       │
│       └── test/
│
└── docs/
```

```
    ├── architecture.md
    ├── api-contract.md
    ├── database.md
    └── agent-flow.md
```

---

4. AI Service

Responsibility

The AI service is responsible for **intelligence**, not money execution.

AI service handles:

- Natural-language buyer requests
- Intent extraction
- LLM reasoning
- Product semantic search
- Embeddings
- Product ranking
- Recommendations
- Upselling
- Tool selection
- Purchase-intent generation
- Agent state

AI service does NOT handle:

- Razorpay secret keys
- Payment execution
- Payment authorization
- Final transaction validation
- Financial limits
- Order ownership
- Payment verification

---

5. AI Agent Flow

Example user request:

```
"I need a black casual shirt under ₹1000."
```

Agent:

```
User Request
     ↓
Intent Extraction
     ↓
Structured Intent
     ↓
Search Catalog
     ↓
Rank Products
     ↓
Recommend
     ↓
User selects product
     ↓
Generate Purchase Intent
     ↓
Send to Spring Boot
```

Example intent:

```
{
  "category": "shirt",
  "color": "black",
  "style": "casual",
  "max_price": 1000
}
```

---

6. AI Tools

The agent should interact with the backend through tools.

Initial tools:

```
search_products()
get_product()
check_inventory()
create_cart()
add_to_cart()
```

```
create_purchase_intent()
get_order_status()
```

Example:

```
AI
 ↓
search_products(
    query="black casual shirt",
    max_price=1000
)
 ↓
Spring Boot API
 ↓
Products
 ↓
AI reasoning
```

---

7. Backend Service

Responsibility

Spring Boot is the **source of truth**.

It controls:

- Merchants
- Products
- Inventory
- Carts
- Orders
- Purchase intents
- Authorization
- Transaction policies
- Razorpay
- Webhooks
- Audit logs

The AI service should never be trusted as the final authority.

---

8. Backend Modules

```
controller/
    REST endpoints

service/
    Business logic

repository/
    Database access

entity/
    Database models

dto/
    API request/response models

razorpay/
    Razorpay integration

policy/
    Transaction safety rules

audit/
    Immutable action history

config/
    Application configuration
```

---

9. Initial API Contract

The two teams should agree on these APIs before implementing the agent.

Product Search

```
POST /api/products/search
```

Request:

```
{
  "query": "black casual shirt",
```

```
  "maxPrice": 1000
}
```

Response:

```
{
  "products": [
    {
      "id": "P102",
      "name": "Black Oversized Shirt",
      "price": 799,
      "currency": "INR",
      "stock": 12
    }
  ]
}
```

Product Details

```
GET /api/products/{productId}
```

Inventory

```
GET /api/products/{productId}/inventory
```

Cart

```
POST /api/cart
```

```
{
  "productId": "P102",
  "quantity": 1
}
```

Purchase Intent

```
POST /api/purchase-intents
```

Request:

```
{
  "productId": "P102",
  "quantity": 1,
  "maxAmount": 1000,
  "reason": "Matches user's request"
}
```

The backend must independently verify:

```
✓ Product exists
✓ Product available
✓ Current price
✓ Quantity
✓ Merchant status
✓ User authorization
✓ Transaction limits
```

---

10. Policy Engine

The policy engine is deterministic.

Example:

```
Maximum transaction amount
Maximum quantity
Require confirmation above amount
Merchant verification
Price-change confirmation
Inventory validation
Duplicate transaction protection
```

Example:

```
AI requests ₹799
      ↓
Policy Engine
      ↓
Amount ≤ limit ✓
Product available ✓
Price unchanged ✓
Authorization ✓
      ↓
ALLOW
```

If:

```
AI requests ₹7,999

User limit = ₹5,000
```

Then:

```
BLOCK

Reason:
Transaction exceeds authorized limit.
```

---

11. Razorpay Layer

Razorpay is accessed **only by Spring Boot**.

```
AI
 ↓
Purchase Intent
 ↓
Spring Boot
 ↓
Policy Engine
 ↓
Razorpay
```

Initial Razorpay integration should cover:

- Test-mode credentials
- Order creation
- Payment flow
- Payment verification
- Payment status
- Webhooks
- Failure handling

Secrets must remain in the backend.

Never expose:

```
RAZORPAY_KEY_SECRET
```

to the AI service or frontend.

---

12. Webhook Flow

```
Razorpay
    │
    │ webhook
    ▼
Spring Boot
    │
    ├── Verify signature
    │
    ├── Validate event
    │
    ├── Update payment
    │
    ├── Update order
    │
    └── Create audit event
```

Example:

```
payment.captured
payment.failed
order.paid
```

13. Audit Trail

Every financial action must be recorded.

Example:

```
{
  "event": "ORDER_CREATED",
  "actor": "AI_BUYER",
  "productId": "P102",
  "amount": 799,
  "policyResult": "PASS",
  "authorizationResult": "PASS",
  "razorpayOrderId": "order_xxx",
  "timestamp": "..."
}
```

The audit trail should answer:

```
What happened?
Who requested it?
Why was it requested?
What did the AI decide?
What policies were checked?
What did Razorpay return?
What happened afterward?
```

14. Database — Initial Entities

Start with:

```
Merchant
Product
ProductVariant
Inventory
User
Cart
CartItem
PurchaseIntent
Order
```

```
Payment
AuditEvent
```

Do NOT build 30 tables initially.

Start with the minimum required for the demo.

---

15. AI ↔ Backend Contract

The most important rule:

```
AI proposes.
Backend validates.
Razorpay executes.
```

Never:

```
AI → Razorpay
```

Always:

```
AI
 ↓
Spring Boot
 ↓
Validation
 ↓
Policy
 ↓
Authorization
 ↓
Razorpay
```

---

16. Failure Handling

We need at least one demonstrable failure.

Example:

```
AI selects product
      ↓
Purchase Intent
      ↓
Policy ✓
      ↓
Razorpay
      ↓
Payment Failed
      ↓
Webhook
      ↓
Spring Boot
      ↓
Update order
      ↓
Audit
      ↓
AI receives failure
```

The system should produce a clear explanation instead of silently failing.

Example:

```
Payment unsuccessful.

Reason:
Payment attempt failed.

Action:
No automatic second transaction was created.

Status:
REQUIRES_USER_ACTION
```

---

17. Development Phases

Phase 1 — Foundation

Backend

- Spring Boot project

- PostgreSQL
- Merchant
- Product
- Product APIs
- Basic authentication

AI

- Python project
- LLM connection
- Basic buyer agent
- Intent extraction

Phase 2 — AI Catalog

Backend

- Product search API
- Inventory API

AI

- Embeddings
- Semantic search
- Product ranking
- Recommendation engine

Phase 3 — Agent Tools

Implement:

```
search_products
get_product
check_inventory
create_cart
add_to_cart
create_purchase_intent
```

Phase 4 — Razorpay

Backend:

```
Purchase Intent
      ↓
Policy
      ↓
Razorpay Order
      ↓
Payment
      ↓
Webhook
```

Phase 5 — Safety

Implement:

- Transaction limits
- Price verification
- Inventory verification
- Authorization
- Duplicate protection
- Audit trail

Phase 6 — Demo & Evaluation

Measure:

```
Catalog search accuracy
Recommendation relevance
Agent task completion
Invalid action blocking
Transaction success rate
Failure handling
```

---

18. MVP

The first working version only needs to demonstrate:

```
1. Merchant adds products
          ↓
```

```
2. AI buyer receives natural-language request
          ↓
3. AI discovers products
          ↓
4. AI recommends product
          ↓
5. User confirms purchase
          ↓
6. Backend validates transaction
          ↓
7. Razorpay test payment
          ↓
8. Webhook confirms result
          ↓
9. Audit trail displayed
```

Everything else comes later.

---

19. Team Split

AI Developer

Own:

```
AI Buyer
LLM
Agent tools
Intent extraction
Embeddings
Semantic search
Recommendations
AI evaluation
```

Backend Developer

Own:

```
Spring Boot
PostgreSQL
REST APIs
Catalog
Cart
```

```
Orders
Policy Engine
Razorpay
Webhooks
Audit
Authentication
```

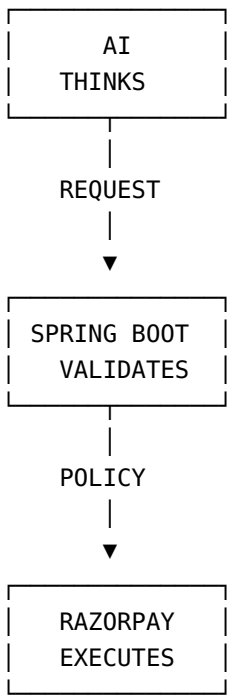
Shared

```
API contract
Database schema
Integration testing
Deployment
Demo
Documentation
Pitch
```

---

20. Golden Rule

The system should always follow:



AI is intelligent but untrusted.

Spring Boot is the source of truth.

Razorpay is the financial execution layer.